Universidad Nacional de Ingeniería
Facultad de Ingeniería Industrial y de Sistemas

IMPACTA

Simulador de Partículas 2D

Lenguaje de programación 1 - SW301U

Docente: Mag. Juan Luis Herencia Guerra

Integrantes:

- Tom Jordan Pérez López
- Raul Antonio Guzmán López
- Ricardo Gaspar Reyes Campos
- Sandro Giancarlo Chavez Vicaña

4 de mayo, Lima 2026

EL CAOS NO ES EL FIN

El desorden aparente es el origen de la información estructurada.

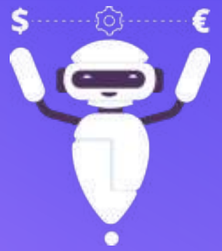


OBJETIVOS



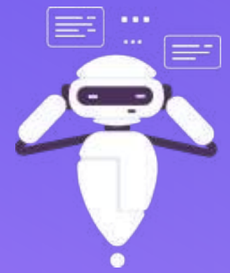
OBJETIVO 01

Simulación de 1000+ partículas con integración de Euler en C.



OBJETIVO 02

Comprobación empírica de la Conservación de Energía.



OBJETIVO 03

Generación de archivos CSV para Ciencia de Datos.

IMPACTA

REQUERIMIENTOS

FUNCIONALES

Motor Físico

Inicialización de N partículas, integración de Euler para movimiento y aplicación de gravedad constante.

Análisis

Cálculo de energía total (cinética/potencial), exportación a CSV y visualización en Python.

Interacciones

Detección de colisiones 2D, resolución elástica (momento) y rebote con paredes (restitución).

NO FUNCIONALES

Implementación

Código modular separado en archivos fuente y cabeceras (entre Python y C).

Gestión de memoria

Manejo dinámico de memoria validado. Reproducibilidad mediante semillas aleatorias fijas.

Optimización y automatización

Escalabilidad hasta 1000 partículas. Sistema de construcción automatizado vía Makefile.

RESTRICCIONES DEL SISTEMA

Espacio 2D rectangular cerrado de dimensiones configurables



Colisiones perfectamente elásticas, pero la fricción (o amortiguamiento viscoso) con el medio será configurable.



Pasos de tiempo (dt) < 0.01s para estabilidad entre tick y tick



La duración de la simulación será definido por el total de ticks



MODELO FÍSICO

SISTEMA

- Partículas puntuales.
- Movimiento en un plano (2D).
- Sin fuerzas externas durante la colisión.
- Interacción solo cuando hay contacto.



SUPUESTOS

- Partículas rígidas
- Colisiones instantáneas.
- No hay rozamiento.
- No hay fuerzas externas (conservativo).
- El espacio es continuo y bidimensional.

CONSERVACIONES

- Se conservan cantidad de movimiento y energía cinética en cada colisión.

Cantidad de movimiento:

$$m_1\vec{v}_1 + m_2\vec{v}_2 = m_1\vec{v}_1' + m_2\vec{v}_2'$$

Energía cinética:

$$\frac{1}{2}m_1v_1^2 + \frac{1}{2}m_2v_2^2 = \frac{1}{2}m_1v_1'^2 + \frac{1}{2}m_2v_2'^2$$

COLISIÓN ELÁSTICA

- La colisión es elástica (e =1).
- Las velocidades después de la colisión se calculan usando las leyes de conservación y la normal al punto de contacto.



DINÁMICA ENTRE COLISIONES

- Cada partícula se mueve con velocidad constante mientras no colisiona.
- Posición actualizada con integración explícita:

$$\vec{r}(t + \Delta t) = \vec{r}(t) + \vec{v}\,\Delta t$$

DETECCIÓN DE COLISIONES

- Se conservan cantidad de movimiento y energía cinética en cada colisión.
- Si hay colisión, se calcula el instante y se actualizan las velocidades.

$$|\vec{r}_1 - \vec{r}_2| \leq R_1 + R_2$$

PARÁMETROS DEL MODELO

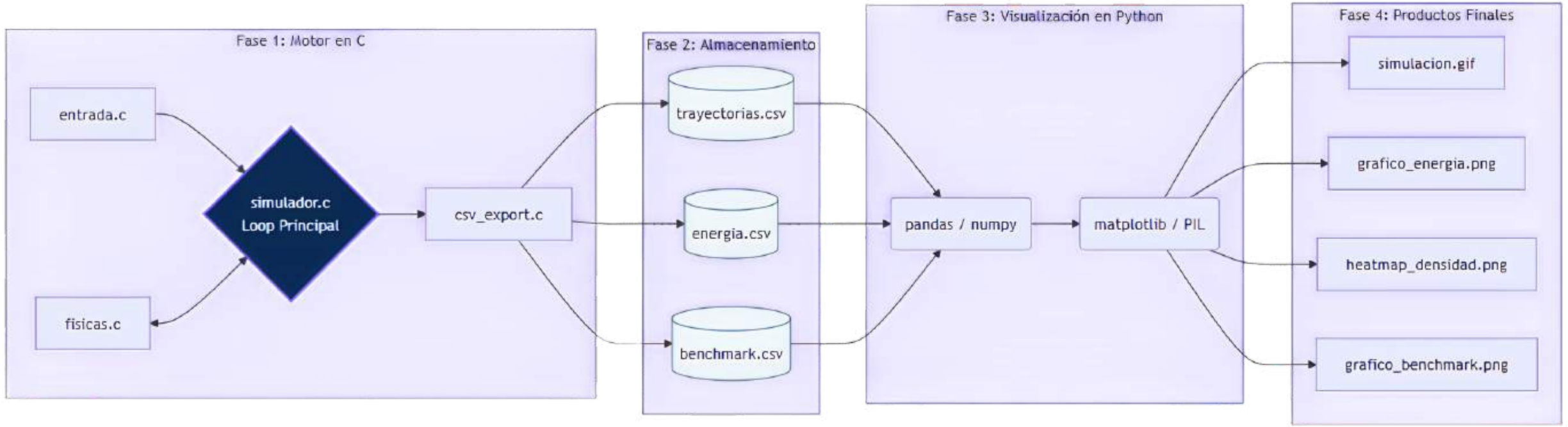
- Masas: m1, m2, m3, ...
- Posiciones iniciales: $\vec{r1}$, $\vec{r2}$, $\vec{r3}$, ...
- Velocidades iniciales: $\vec{v1}$, $\vec{v2}$, $\vec{v3}$, ...
- Radios: R1, R2, ...
- Coeficiente de restitución: e = 1 (elástico)
- Paso de tiempo: Δt

IMPACTA

FASES DEL PROGRAMA







MÓDULOS EN C

MÓDULOS EN PYTHON

NOTA: ELABORADO CON MERMAID.

MÓDULOS EN C



Entradas.c

Menú/configuración

Físicas.c

Propiedades fisicas y colisiones

Simulador.C

Simulacion de aprticulas con, hashing espacial

csv_export.c

Generación de gráficos

1

Entrada

Ingreso de parámetros

2

Inizialización

Crear particulas más hash tabla

3

Simulación

Loop de tiks

4

Cálculo

Energía Ek, Ep, etc por tiks

5

Salida

Archivos csv

IMPACTA

MÓDULOS EN PYTHON



Pandas

Manejo de archivos csv

Numpy

Operaciones númericas

Matplotlib

Generación de gráficos



1

Entrada

Lectura de CSVs generados en C.

2

Proceso

Pandas y Numpy limpian datos.

3

Análisis

Animación de curvas de energía, heatmap, benchmark con Matplotlib

4

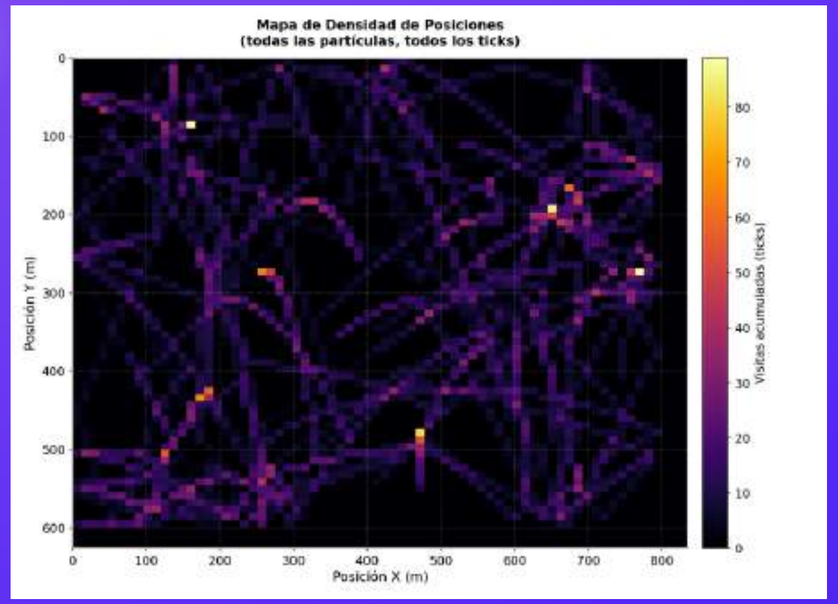
Salida

Exportación de imágenes

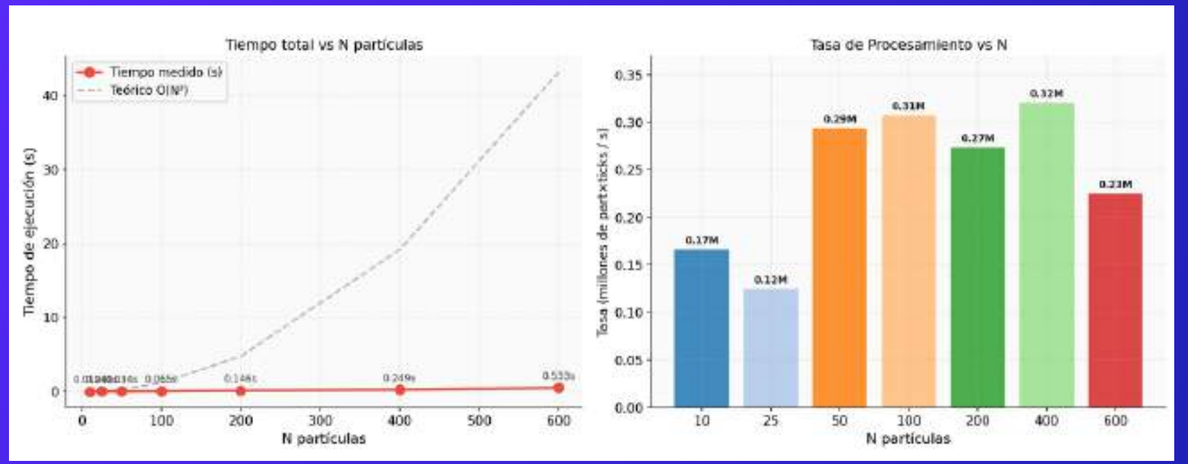
IMPACTA

ANÁLISIS DE DATOS

MAPA DE CALOR



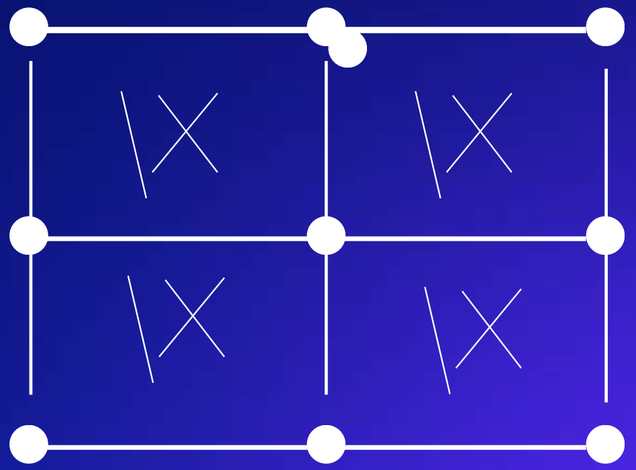
BENCHMARK DE RENDIMIENTO



ANÁLISIS DE COMPLEJIDAD



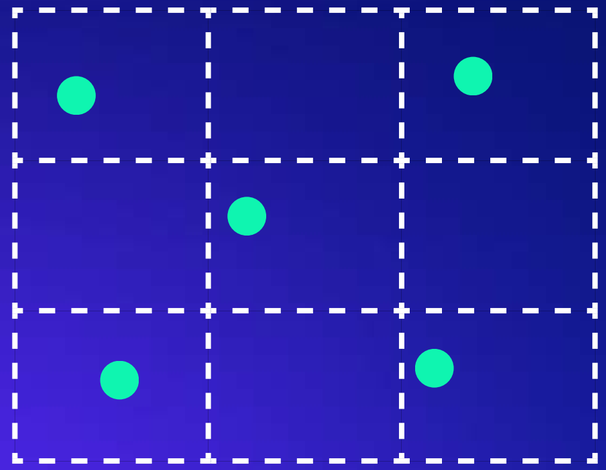
REDUCCIÓN DE COMPLEJIDAD: O(N²) → O(N)



Cada particulas vs todas

N·(N-1)/2 comparaciones

O(N²)



Spatial Hashing en la fase de colisiones

~5N comparaciones

O(N)



COMPARACIÓN DE COMPLEJIDAD

Bucle B: Force — O(N²)

```
for (int i = 0; i < N; i++)
        for (int j = i+1; j < N; j++)
                resolver_colision(&p[i], &p[j]);
```

Bucle B: Spatial Hashing — O(N)

```
memset(head, -1, hash_size * sizeof(int)); // limpiar
for (int i = 0; i < N; i++) { // llenar
        int h = hash(p[i].x/L, p[i].y/L, hash_size);
        next[i] = head[h]; head[h] = i;
}
for (int i = 0; i < N; i++) // revisar
        revisar_celda_propia_y_4_vecinas(i); // K ≈ 5
```

"En vez de comparar contra todas las partículas del universo, cada partícula solo verifica contra sus vecinos inmediatos en la cuadrícula."

ARQUITECTURA Y DIAGRAMAS



IMPACTA

DIAGRAMA DE ESTRUCTURAS

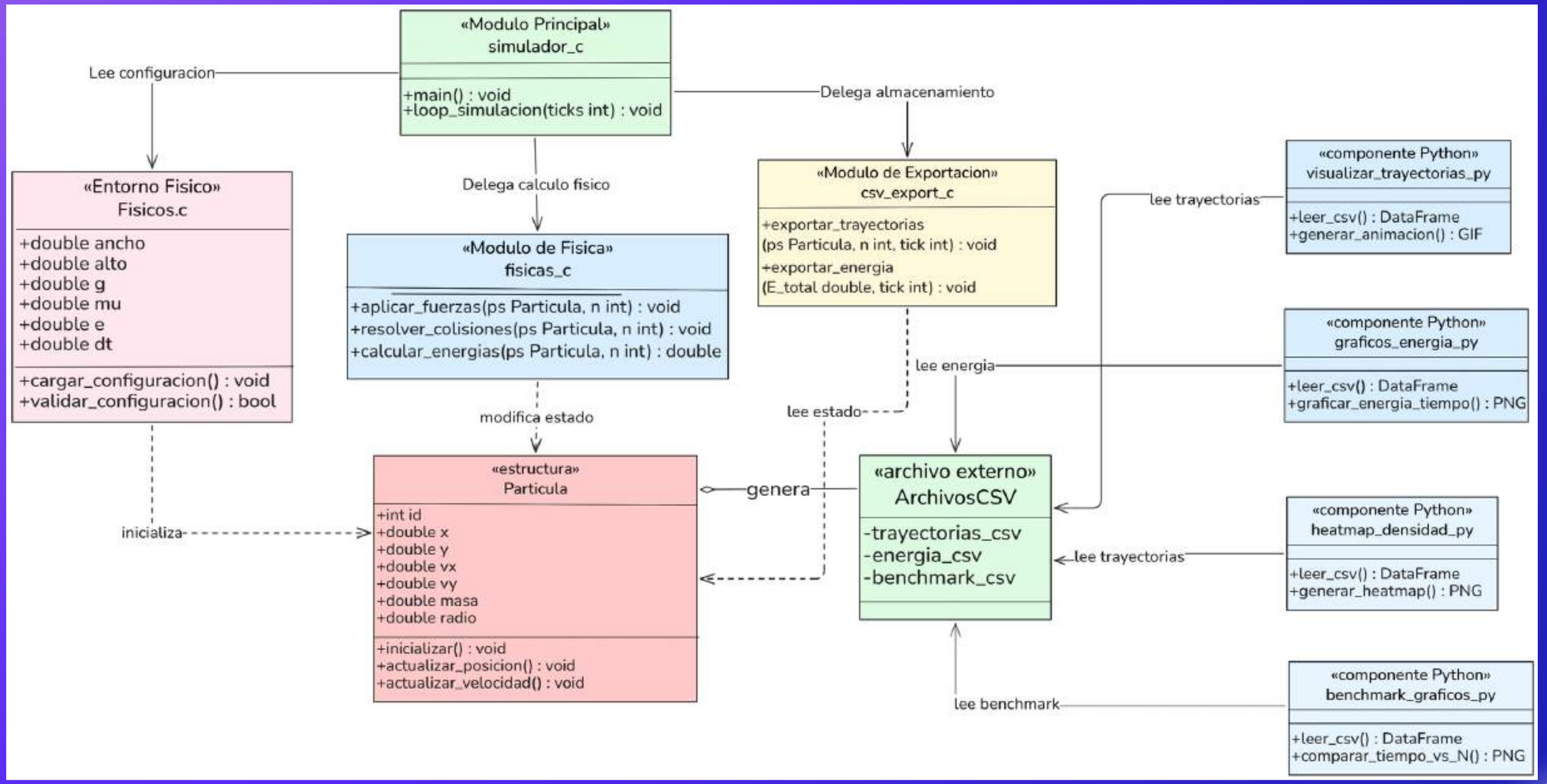


DIAGRAMA DE CASOS DE USO

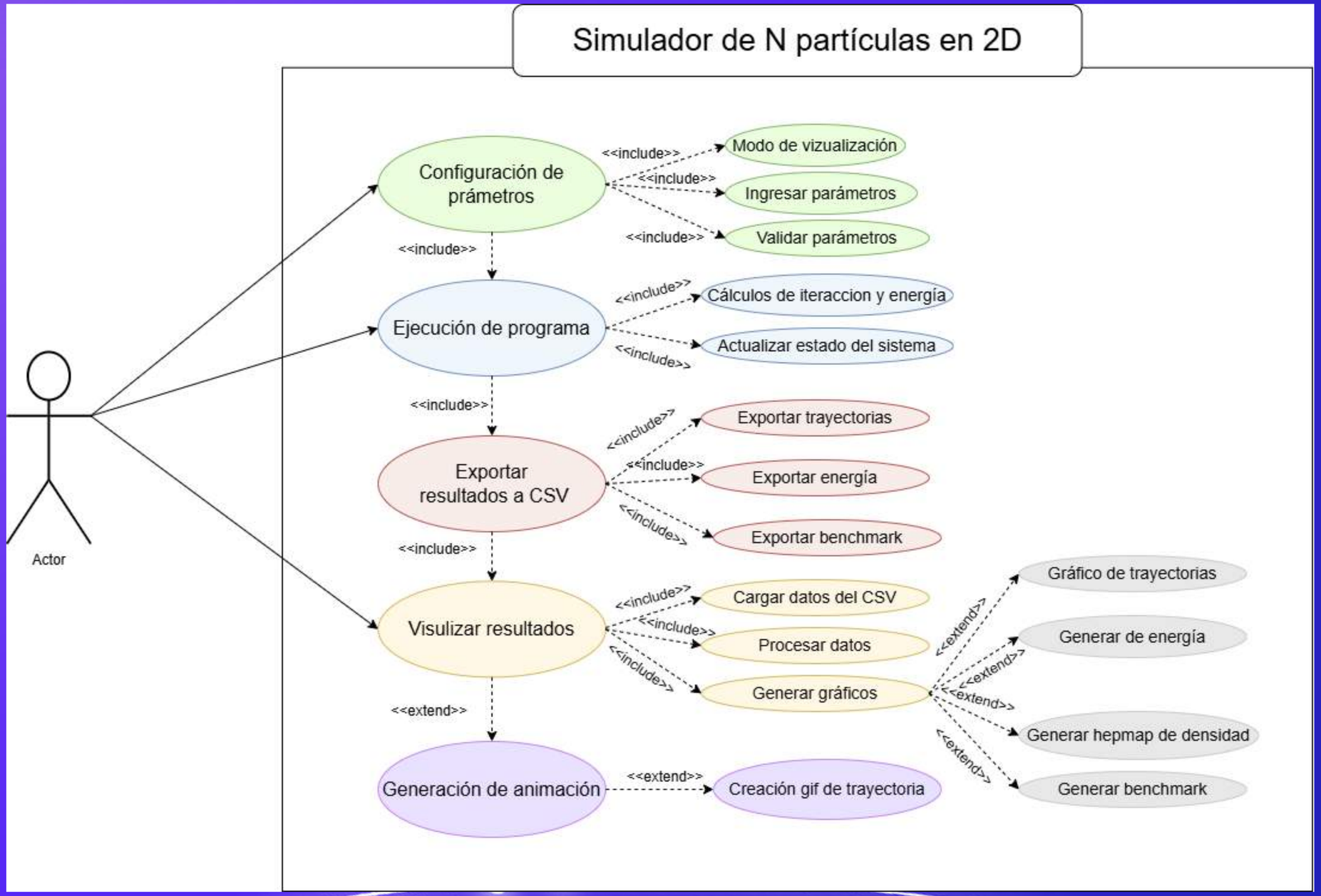


DIAGRAMA DE SECUENCIAS

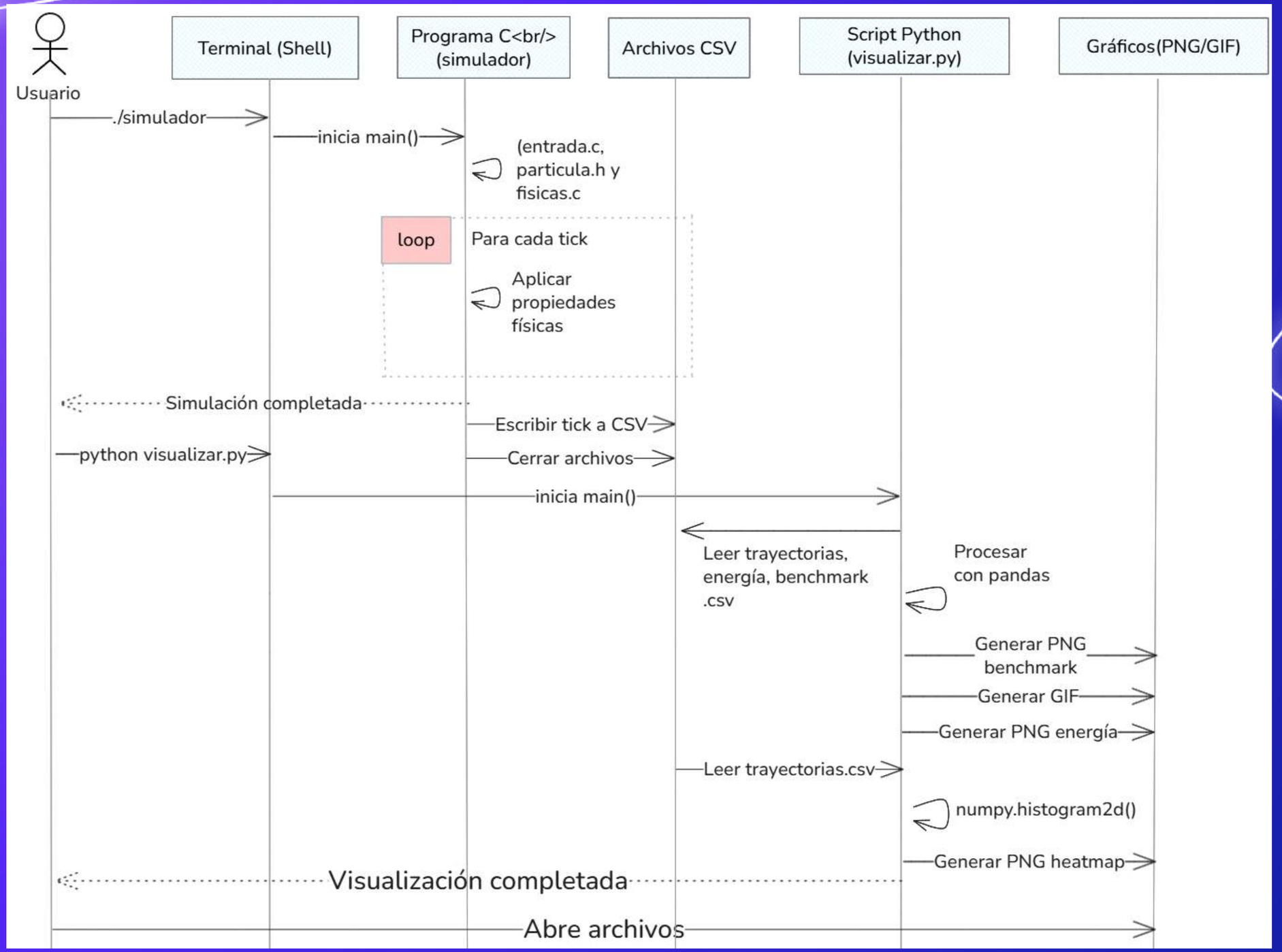
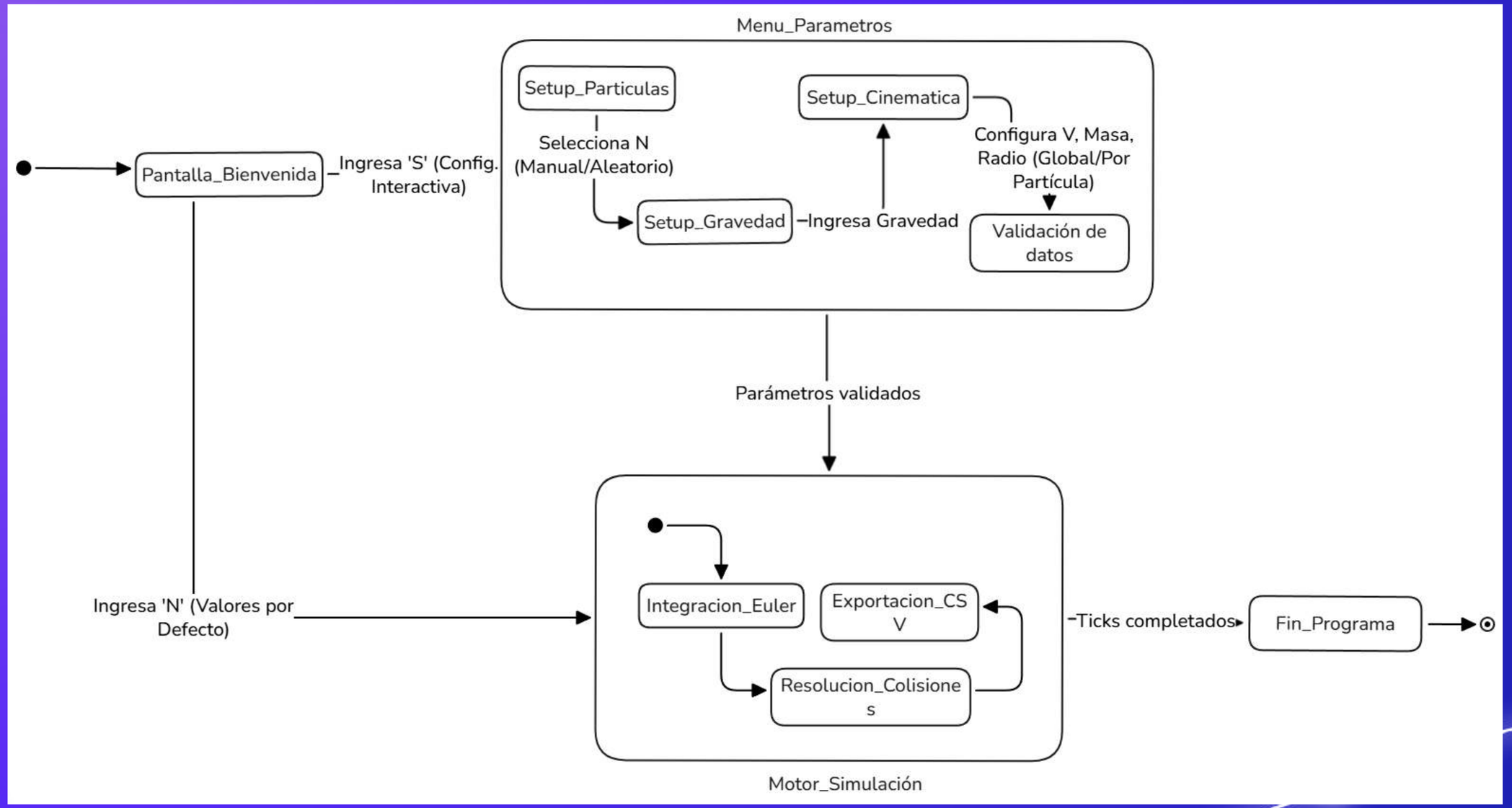


DIAGRAMA DE INTERFAZ DE USUARIO (TUI)



MUCHAS GRACIAS

IMPACTA



PÁGINA DE RECURSOS

